

POC-03

Real-Time Data Platform with Event Hubs, Databricks, Fabric and Synapse

Beginner Implementation, Troubleshooting, Recovery & Verification Guide

Built from your actual POC evidence: This guide is reconstructed from poc3_setup_challenges_step.md, the supplied screenshot archive, and the POC project assets. It preserves the errors you hit, the fixes that worked, and the final verified Databricks path.

Security: A storage access key was visible in one supplied screenshot. It has been redacted in this PDF. If that key was ever exposed outside your private environment, rotate it in Azure and update dependent applications.

Prepared: 03 September 2026

Contents

1. What this POC builds and what was actually validated
 2. Architecture and project structure
 3. First-time local setup and command runbook
 4. Azure resource creation: Resource Group, Event Hubs, Storage, Databricks
 5. Environment configuration (.env) and Event Hubs verification
 6. Databricks Serverless implementation and all troubleshooting
 7. Silver validation, Gold creation, and Databricks SQL serving
 8. Synapse Serverless SQL reference and ADLS requirement
 9. Fabric Eventstream / Eventhouse / KQL reference path
 10. Monitoring, failure recovery, evidence, and cleanup
 11. Fast rerun checklist for future practice
 12. Issue-to-fix quick reference
- Appendix A. Full screenshot evidence index

1. What this POC builds and what was actually validated

The original POC is a near-real-time shipment analytics platform. A Python producer sends shipment events to Azure Event Hubs. The planned architecture then fans out to Azure Databricks for lakehouse processing and to Microsoft Fabric Real-Time Intelligence for KQL analytics, with curated data served through Synapse serverless SQL and semantic models.

Original POC architecture

```

Python synthetic producer
|
v
Azure Event Hubs
|
+--> Databricks / Structured Streaming
|   +--> Bronze
|   +--> Silver
|   +--> Gold Parquet
|
+--> Fabric Eventstream
    +--> Eventhouse / KQL

Gold data
+--> Synapse serverless SQL
+--> Fabric / semantic model
  
```

Important actual-run difference: The successful Databricks run shown in your screenshots uses Serverless compute and a Unity Catalog managed Volume at /Volumes/dbw_logistics_poc/default/realtime. The standalone ADLS storage account was created, but the final Bronze/Silver/Gold evidence in the screenshots is inside the Databricks Volume, not the external ADLS container.

Component	Status	Evidence	What the evidence shows
Resource group	Validated	Screenshot 1	Resource group created.

Component	Status	Evidence	What the evidence shows
Event Hubs namespace	Validated	Screenshot 2	Namespace deployment completed; event producer/consumer later proved connectivity.
Storage account	Validated	Screenshots 3, 9	Storage account created; access-key page captured. Keys are masked/redacted in this guide.
Local Event Hub producer	Validated	Screenshots 4, 5	Initial malformed connection-string failure, then successful event sends.
Local Event Hub receiver	Validated	Screenshot 6	Events read from Event Hubs.
Azure Databricks workspace	Validated	Screenshot 7	Workspace deployment completed.
Databricks Bronze ingestion	Validated	Screenshot 11	Serverless-compatible available-now batch wrote 50 rows.
Unity Catalog Volume	Validated	Screenshot 11 (catalog)	Delta log visible under /Volumes/dbw_logistics_poc/default/realtime.
Silver curated layer	Validated after recovery	Screenshots 12, 13	Initial Silver count was 0; recovery populated 48 clean rows with 0 duplicate event IDs.
Gold Parquet/KPIs	Validated	Screenshot 14	Gold KPI summary exported from 48 clean Silver events.
Databricks SQL serving	Validated	Screenshots 15, 16	Views over Silver Delta and Gold Parquet were created in Databricks SQL.
Fabric Eventstream/Eventhouse/KQL	Reference in project	No supplied screenshot	The project contains KQL/Fabric setup assets, but the supplied evidence does not prove this path was completed.
Synapse serverless SQL	Reference in project	No supplied screenshot	SQL script is included, but supplied screenshots show Databricks SQL rather than Synapse Studio.
Purview / semantic model	Reference in project	No supplied screenshot	Design/scripts are available; completion is not evidenced by the supplied screenshots.

2. Architecture and project structure

2.1 Event schema

Shipment event example

```
{
  "event_id": "evt-001",
  "order_id": 1001,
  "event_type": "SHIPPED",
  "event_ts": "2026-08-28T10:00:00Z",
  "region": "IN-SOUTH",
  "revenue": 1299.50,
  "fulfillment_minutes": 120,
  "producer_seq": 1
}
```

2.2 Project structure

```
POC_03_REALTIME_FABRIC_SYNAPSE/
├── .env.example
├── requirements.txt
├── producer/
│   ├── send_events.py
│   └── receive_events.py
├── databricks/
│   ├── stream_to_delta.py
│   ├── inspect_and_validate.py
│   └── export_gold_parquet.py
├── fabric/setup_notes.md
├── kql/
│   ├── create_shipment_table.kql
│   └── shipment_queries.kql
├── synapse/serverless_queries.sql
├── semantic_model/measures.dax
├── governance/purview_notes.md
├── monitoring/monitoring_checklist.md
├── docs/
│   ├── azure_portal_setup.md
│   ├── troubleshooting.md
│   └── cleanup.md
└── evidence/
```

3. First-time local setup and command runbook

Use this order on a clean Windows machine. Run commands from the project root unless a step says to use Azure Portal or Databricks.

3.1 Create or activate Python environment

Windows Command Prompt / PowerShell

```
cd <path-to-project>\POC_03_REALTIME_FABRIC_SYNAPSE
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
copy .env.example .env
```

If you already use a shared Python environment: Activate that environment instead of creating .venv, then run `pip install -r requirements.txt`. Your screenshots show the POC was executed from an existing common Python environment.

3.2 First connectivity test

```
python producer\send_events.py --count 20 --delay 0.2
python producer\receive_events.py --max-events 10 --starting-position -1
```

3.3 Main batch used for Databricks

Serverless available-now timing rule: For the final Serverless pattern, send the Event Hub batch BEFORE starting the Databricks availableNow run. availableNow processes the data available when the query starts and then finishes.

```
python producer\send_events.py --count 50
```

Then run the Databricks notebooks in this order:

1. `stream_to_delta.py`
2. `inspect_and_validate.py`
3. `export_gold_parquet.py`

4. Azure resource creation - beginner portal steps

4.1 Create the Resource Group

4. Open Azure Portal and search for Resource groups.
5. Select Create.
6. Choose your active subscription.
7. Create/select the POC resource group. Your run used `rg-logistics-poc`.
8. Choose one region where practical so related resources stay close.
9. Select Review + create, then Create.

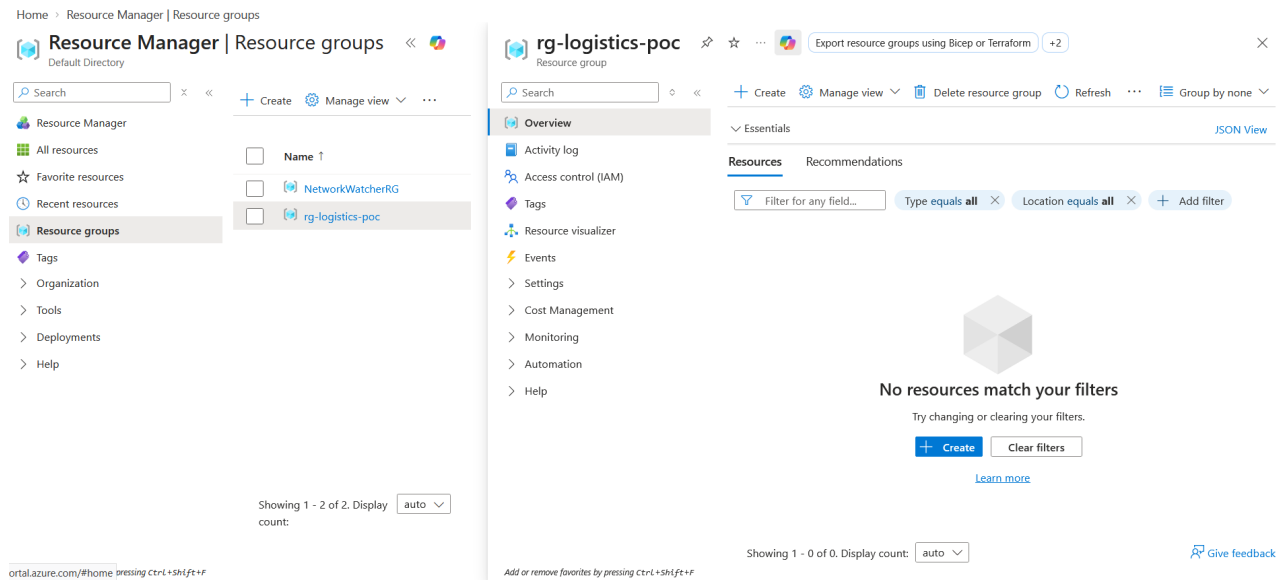


Figure 1 - Resource group created successfully.

4.2 Create Azure Event Hubs namespace and Event Hub

10. In Azure Portal search for Event Hubs and select Create namespace.
11. Use the same resource group.
12. Choose a globally unique namespace name. The successful Databricks code later used the example namespace eh-logistics-poc-dev.
13. Choose the smallest suitable tier/capacity for this learning POC.
14. After deployment, open the namespace.
15. Under Entities > Event Hubs, create an Event Hub named shipment-events.
16. Keep a small/default partition count for the POC.
17. Under Shared access policies, use/create a policy with Send and Listen rights for this beginner connection-string test.
18. Copy Connection string-primary key - not only the primary key - into your local .env.

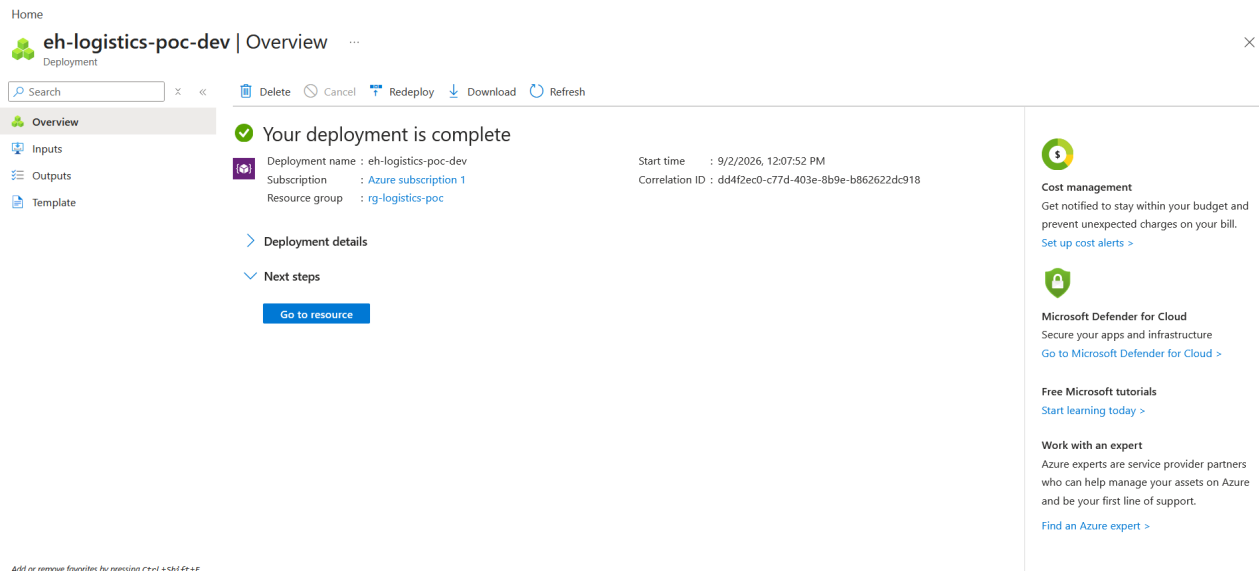


Figure 2 - Event Hubs namespace deployment completed.

4.3 Create ADLS Gen2 storage account and realtime container

19. Search Azure Portal for Storage accounts and select Create.
20. Use the POC resource group.
21. Choose a globally unique lowercase storage account name.
22. Performance: Standard; redundancy: low-cost option appropriate for a POC.
23. Enable Hierarchical namespace so the account supports ADLS Gen2 semantics.
24. After deployment, open Data storage > Containers.
25. Create a private container named realtime.

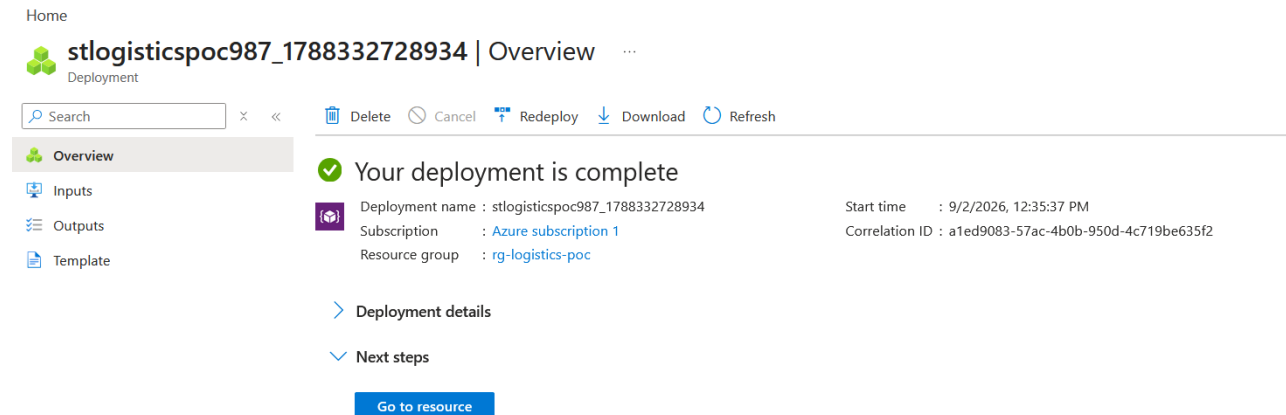


Figure 3 - Storage account deployment completed.

4.4 Storage access key page - what it was used for and why it failed on Serverless

The troubleshooting path temporarily used the storage account access key with `spark.conf.set(...)`. The Portal location is Storage account > Security + networking > Access keys.

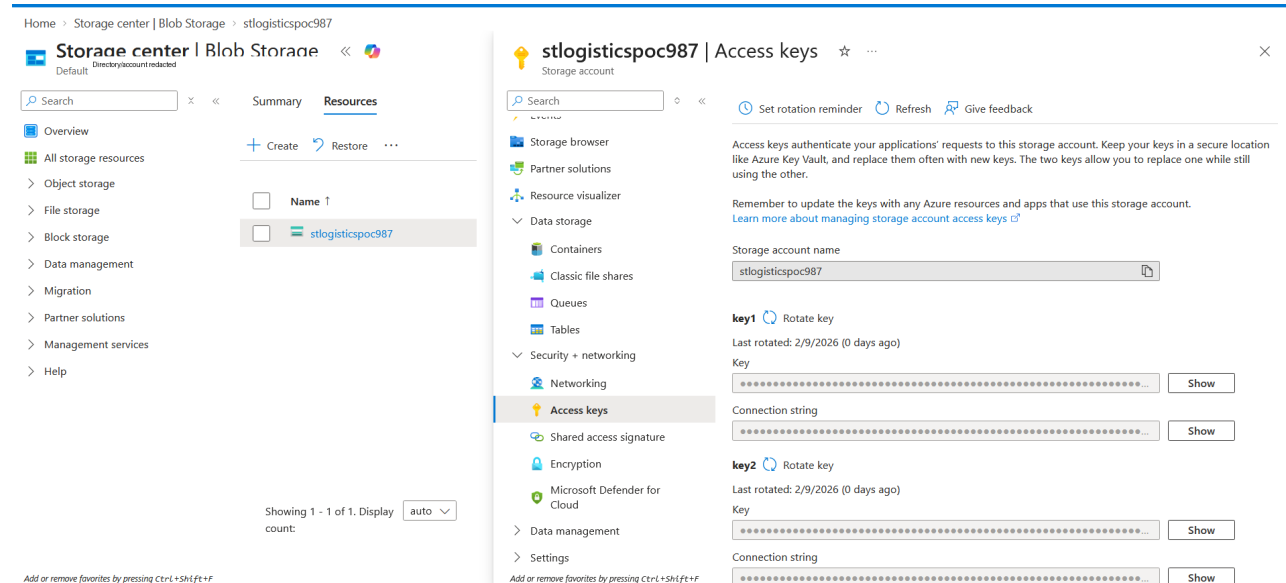


Figure 4 - Storage account access-key page. Key values are masked by Azure; account identity was additionally redacted in this guide.

Do not use the access key in source code for the final design: The direct key approach later failed on Databricks Serverless and one screenshot exposed a key in notebook code. Prefer Unity Catalog storage credentials/managed identity or a Databricks secret. Rotate any exposed key.

4.5 Create Azure Databricks workspace

26. Search Azure Portal for Azure Databricks and select Create.
27. Use rg-logistics-poc.
28. Workspace name used in the run: dbw-logistics-poc.
29. Deploy, then select Launch Workspace.
30. Your final successful run used Databricks Serverless compute, so the guide below follows the Serverless-compatible path.

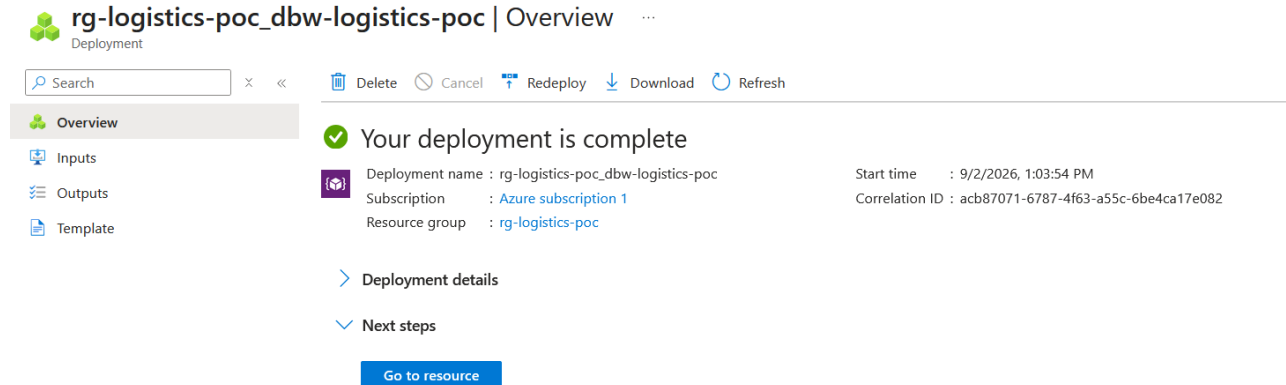


Figure 5 - Azure Databricks workspace deployment completed.

5. Configure .env and verify Event Hubs locally

5.1 Values to collect

Variable	Where it comes from
EVENT_HUB_CONNECTION_STRING	Event Hubs namespace > Shared access policies > policy > Connection string-primary key.
EVENT_HUB_NAME	shipment-events
EVENT_HUB_CONSUMER_GROUP	\$Default for the beginner test.
EVENT_HUB_NAMESPACE	The namespace name only, for example eh-logistics-poc-dev.
EVENT_HUB_KAFKA_BOOTSTRAP	<namespace>.servicebus.windows.net:9093
ADLS_ACCOUNT_NAME	Your storage account name.
ADLS_CONTAINER_NAME	realtime

Safe template - never put a real secret in Git


```

EVENT_HUB_CONNECTION_STRING=Endpoint=sb://
<namespace>.servicebus.windows.net;/SharedAccessKeyName=<policy>;SharedAccessKey=<secret>
EVENT_HUB_NAME=shipment-events
EVENT_HUB_CONSUMER_GROUP=$Default
EVENT_COUNT=200
EVENT_DELAY_SECONDS=0.15
DUPLICATE_EVERY=25
LATE_EVENT_EVERY=40
ADLS_ACCOUNT_NAME=<storage-account-name>
ADLS_CONTAINER_NAME=realtime
DELTA_BRONZE_PATH=abfss://realtime@<storage-account-name>.dfs.core.windows.net/delta/bronze/shipment_events
DELTA_SILVER_PATH=abfss://realtime@<storage-account-name>.dfs.core.windows.net/delta/silver/shipment_events
GOLD_PARQUET_PATH=abfss://realtime@<storage-account-name>.dfs.core.windows.net/gold/shipment_summary
EVENT_HUB_NAMESPACE=<namespace>
EVENT_HUB_KAFKA_BOOTSTRAP=<namespace>.servicebus.windows.net:9093

```

5.2 Issue 1 - send_events.py failed: connection string blank or malformed

```

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse>python producer/send_events.py
Traceback (most recent call last):
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse\producer\send_events.py", line 141, in <module>
    main()
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse\producer\send_events.py", line 137, in main
    send_events(args.count, args.delay, args.duplicate_every, args.late_every)
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse\producer\send_events.py", line 70, in send_events
    producer = EventHubProducerClient.from_connection_string(
  File "C:\Users\ermdi\projects\common_venv_py312\venv\Lib\site-packages\azure\eventhub\_producer_client.py", line 545, in from_connection_string
    constructor_args = cls._from_connection_string(
  File "C:\Users\ermdi\projects\common_venv_py312\venv\Lib\site-packages\azure\eventhub\_client_base.py", line 329, in _from_connection_string
    host, policy, key, entity, token, token_expiry, emulator = _parse_conn_str(conn_str, **kwargs)
  File "C:\Users\ermdi\projects\common_venv_py312\venv\Lib\site-packages\azure\eventhub\_client_base.py", line 88, in _parse_conn_str
    conn_settings = core.parse_connection_string(conn_str, case_sensitive_keys=check_case)
  File "C:\Users\ermdi\projects\common_venv_py312\venv\Lib\site-packages\azure\core\utils\_connection_string_parser.py", line 27, in parse_connection_string
    raise ValueError("Connection string is either blank or malformed.")
ValueError: Connection string is either blank or malformed.
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse>

```

Figure 6 - Initial producer failure: ValueError - connection string is either blank or malformed.

The run log records this as the first major local issue. Check the following in order:

31. Make sure you copied Connection string-primary key, not only Primary key.
32. Confirm the .env file is named exactly .env, not .env.txt.
33. Do not put spaces around the variable name or accidentally comment it out.
34. Confirm the value starts with Endpoint=sb://.
35. Check what Python actually reads before rerunning the producer.

Diagnosis and retry commands

```

Get-ChildItem -Hidden .env*
python -c "import os; from dotenv import load_dotenv; load_dotenv(); print('Length:', len(os.getenv('EVENT_HUB_CONNECTION_STRING') or ''))"
python producer/send_events.py

```

```
[168/200] NORMAL id=evt-9a739fb158ce type=SHIPPED region=IN-WEST ts=2026-09-02T07:19:04Z
[169/200] NORMAL id=evt-4389556967d2 type=PACKED region=IN-EAST ts=2026-09-02T07:19:05Z
[170/200] NORMAL id=evt-a9576821da9e type=DELAYED region=IN-WEST ts=2026-09-02T07:19:05Z
[171/200] NORMAL id=evt-037c0b970c9b type=SHIPPED region=IN-NORTH ts=2026-09-02T07:19:06Z
[172/200] NORMAL id=evt-7eb72f74fee9 type=PACKED region=IN-EAST ts=2026-09-02T07:19:06Z
[173/200] NORMAL id=evt-a48492969b00 type=CREATED region=IN-WEST ts=2026-09-02T07:19:06Z
[174/200] NORMAL id=evt-afc9f3a5cbc7 type=DELIVERED region=IN-EAST ts=2026-09-02T07:19:07Z
[175/200] DUPLICATE id=evt-afc9f3a5cbc7 type=DELIVERED region=IN-EAST ts=2026-09-02T07:19:07Z
[176/200] NORMAL id=evt-b1494312d311 type=SHIPPED region=IN-SOUTH ts=2026-09-02T07:19:07Z
[177/200] NORMAL id=evt-0547d6932fb3 type=DELAYED region=IN-NORTH ts=2026-09-02T07:19:08Z
[178/200] NORMAL id=evt-83806a05ec90 type=CREATED region=IN-SOUTH ts=2026-09-02T07:19:08Z
[179/200] NORMAL id=evt-577d1f2ad115 type=DELIVERED region=IN-EAST ts=2026-09-02T07:19:09Z
[180/200] NORMAL id=evt-a17fa52f752e type=PACKED region=IN-SOUTH ts=2026-09-02T07:19:09Z
[181/200] NORMAL id=evt-369c9c532d4f type=IN_TRANSIT region=IN-WEST ts=2026-09-02T07:19:09Z
[182/200] NORMAL id=evt-08bcf099bbbf type=IN_TRANSIT region=IN-NORTH ts=2026-09-02T07:19:10Z
[183/200] NORMAL id=evt-6f04c50ba215 type=PACKED region=IN-SOUTH ts=2026-09-02T07:19:10Z
[184/200] NORMAL id=evt-1d20a96e32af type=PACKED region=IN-EAST ts=2026-09-02T07:19:11Z
[185/200] NORMAL id=evt-95975e9ad9bb type=DELAYED region=IN-WEST ts=2026-09-02T07:19:11Z
[186/200] NORMAL id=evt-a3309418b466 type=PACKED region=IN-NORTH ts=2026-09-02T07:19:11Z
[187/200] NORMAL id=evt-633a01df7738 type=DELIVERED region=IN-NORTH ts=2026-09-02T07:19:12Z
[188/200] NORMAL id=evt-ee02e20ac5b0 type=PACKED region=IN-EAST ts=2026-09-02T07:19:12Z
[189/200] NORMAL id=evt-a5161ac6a187 type=SHIPPED region=IN-SOUTH ts=2026-09-02T07:19:13Z
[190/200] NORMAL id=evt-184820e1a25f type=DELIVERED region=IN-WEST ts=2026-09-02T07:19:13Z
[191/200] NORMAL id=evt-8073c63e6ecd type=SHIPPED region=IN-NORTH ts=2026-09-02T07:19:13Z
[192/200] NORMAL id=evt-d267c573fea8 type=SHIPPED region=IN-WEST ts=2026-09-02T07:19:14Z
[193/200] NORMAL id=evt-93f37ab95dad type=PACKED region=IN-SOUTH ts=2026-09-02T07:19:14Z
[194/200] NORMAL id=evt-000b2a08bd99 type=CREATED region=IN-WEST ts=2026-09-02T07:19:15Z
[195/200] NORMAL id=evt-0264ddf8604e type=DELIVERED region=IN-SOUTH ts=2026-09-02T07:19:15Z
[196/200] NORMAL id=evt-c264682cdf3f type=SHIPPED region=IN-EAST ts=2026-09-02T07:19:15Z
[197/200] NORMAL id=evt-4ed1aa278289 type=DELIVERED region=IN-WEST ts=2026-09-02T07:19:16Z
[198/200] NORMAL id=evt-5ecaabd51a4e type=SHIPPED region=IN-SOUTH ts=2026-09-02T07:19:16Z
[199/200] NORMAL id=evt-94918d3c3d33 type=PACKED region=IN-EAST ts=2026-09-02T07:19:17Z
[200/200] DUPLICATE id=evt-94918d3c3d33 type=PACKED region=IN-EAST ts=2026-09-02T07:19:17Z
```

Summary
 sent attempts : 200
 duplicates : 8
 late events : 4

Check Event Hubs > Monitoring > Metrics > Incoming Messages.

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse>

Figure 7 - Producer succeeded after fixing the .env connection string; 200 events were sent.

5.3 Verify Event Hubs with receive_events.py

```
python producer\receive_events.py --max-events 20 --starting-position -1
```

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse>python producer\receive_events.py
Receiving up to 20 events from shipment-events (consumer group=Default, starting_position=-1)
[001] partition=0 offset=0 sequence=0 payload={'event_id': 'evt-8ef4eaaec6ff', 'order_id': 1003, 'event_type': 'CREATED', 'event_ts': '2026-09-02T07:17:58Z', 'region': 'IN-WEST', 'revenue': 2977.2, 'fulfillment_minutes': 720, 'producer_seq': 3}
[002] partition=0 offset=296 sequence=1 payload={'event_id': 'evt-45ef66cf89bd', 'order_id': 1004, 'event_type': 'PACKED', 'event_ts': '2026-09-02T07:17:59Z', 'region': 'IN-WEST', 'revenue': 774.51, 'fulfillment_minutes': 281, 'producer_seq': 4}
[003] partition=0 offset=592 sequence=2 payload={'event_id': 'evt-3fca5b1a4b08', 'order_id': 1006, 'event_type': 'DELIVERED', 'event_ts': '2026-09-02T07:18:00Z', 'region': 'IN-WEST', 'revenue': 1106.75, 'fulfillment_minutes': 463, 'producer_seq': 6}
[004] partition=1 offset=0 sequence=0 payload={'event_id': 'evt-2ca2fc7fc93b', 'order_id': 1001, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:17:55Z', 'region': 'IN-EAST', 'revenue': 300.13, 'fulfillment_minutes': 304, 'producer_seq': 1}
[005] partition=0 offset=896 sequence=3 payload={'event_id': 'evt-96231f2a0c62', 'order_id': 1008, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:18:01Z', 'region': 'IN-WEST', 'revenue': 571.77, 'fulfillment_minutes': 436, 'producer_seq': 8}
[006] partition=1 offset=296 sequence=1 payload={'event_id': 'evt-464de99e4d1e', 'order_id': 1002, 'event_type': 'DELIVERED', 'event_ts': '2026-09-02T07:17:58Z', 'region': 'IN-SOUTH', 'revenue': 2189.46, 'fulfillment_minutes': 319, 'producer_seq': 2}
[007] partition=0 offset=1192 sequence=4 payload={'event_id': 'evt-d10e5fda2a7c', 'order_id': 1010, 'event_type': 'CREATED', 'event_ts': '2026-09-02T07:18:01Z', 'region': 'IN-NORTH', 'revenue': 3766.99, 'fulfillment_minutes': 557, 'producer_seq': 10}
[008] partition=1 offset=600 sequence=2 payload={'event_id': 'evt-b34bdc0a443c', 'order_id': 1005, 'event_type': 'CREATED', 'event_ts': '2026-09-02T07:17:59Z', 'region': 'IN-SOUTH', 'revenue': 3365.32, 'fulfillment_minutes': 183, 'producer_seq': 5}
[009] partition=0 offset=1496 sequence=5 payload={'event_id': 'evt-3b2a6e307127', 'order_id': 1012, 'event_type': 'IN_TRANSIT', 'event_ts': '2026-09-02T07:18:02Z', 'region': 'IN-WEST', 'revenue': 651.76, 'fulfillment_minutes': 609, 'producer_seq': 12}
[010] partition=1 offset=904 sequence=3 payload={'event_id': 'evt-fc614d256ea0', 'order_id': 1007, 'event_type': 'PACKED', 'event_ts': '2026-09-02T07:18:00Z', 'region': 'IN-SOUTH', 'revenue': 3497.85, 'fulfillment_minutes': 591, 'producer_seq': 7}
[011] partition=0 offset=1800 sequence=6 payload={'event_id': 'evt-aa9f7787cdeb', 'order_id': 1014, 'event_type': 'DELIVERED', 'event_ts': '2026-09-02T07:18:03Z', 'region': 'IN-WEST', 'revenue': 539.8, 'fulfillment_minutes': 707, 'producer_seq': 14}
[012] partition=1 offset=1208 sequence=4 payload={'event_id': 'evt-93c641182a57', 'order_id': 1009, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:18:01Z', 'region': 'IN-SOUTH', 'revenue': 290.44, 'fulfillment_minutes': 272, 'producer_seq': 9}
[013] partition=0 offset=2104 sequence=7 payload={'event_id': 'evt-011f92bc7c5f', 'order_id': 1015, 'event_type': 'PACKED', 'event_ts': '2026-09-02T07:18:03Z', 'region': 'IN-WEST', 'revenue': 307.1, 'fulfillment_minutes': 495, 'producer_seq': 15}
[014] partition=1 offset=1512 sequence=5 payload={'event_id': 'evt-ff3b71f750a7', 'order_id': 1011, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:18:02Z', 'region': 'IN-SOUTH', 'revenue': 4907.92, 'fulfillment_minutes': 433, 'producer_seq': 11}
[015] partition=0 offset=2400 sequence=8 payload={'event_id': 'evt-a0b30359711f', 'order_id': 1016, 'event_type': 'DELIVERED', 'event_ts': '2026-09-02T07:18:04Z', 'region': 'IN-WEST', 'revenue': 625.73, 'fulfillment_minutes': 406, 'producer_seq': 16}
[016] partition=1 offset=1816 sequence=6 payload={'event_id': 'evt-2a3597cbc284', 'order_id': 1013, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:18:03Z', 'region': 'IN-SOUTH', 'revenue': 697.03, 'fulfillment_minutes': 711, 'producer_seq': 13}
[017] partition=0 offset=2704 sequence=9 payload={'event_id': 'evt-66ac2fdad98b', 'order_id': 1018, 'event_type': 'IN_TRANSIT', 'event_ts': '2026-09-02T07:18:05Z', 'region': 'IN-WEST', 'revenue': 4158.48, 'fulfillment_minutes': 573, 'producer_seq': 18}
[018] partition=1 offset=2120 sequence=7 payload={'event_id': 'evt-703eb2169fe2', 'order_id': 1017, 'event_type': 'PACKED', 'event_ts': '2026-09-02T07:18:04Z', 'region': 'IN-SOUTH', 'revenue': 2579.07, 'fulfillment_minutes': 279, 'producer_seq': 17}
[019] partition=0 offset=3008 sequence=10 payload={'event_id': 'evt-be2b56fe50f', 'order_id': 1021, 'event_type': 'CREATED', 'event_ts': '2026-09-02T07:18:06Z', 'region': 'IN-NORTH', 'revenue': 265.54, 'fulfillment_minutes': 202, 'producer_seq': 21}
[020] partition=1 offset=2424 sequence=8 payload={'event_id': 'evt-5c2c99abc2e', 'order_id': 1019, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:18:04Z', 'region': 'IN-SOUTH', 'revenue': 2579.07, 'fulfillment_minutes': 279, 'producer_seq': 17}
```

Figure 8 - Local receiver reading Event Hub messages successfully.

Pass condition: Do not continue to Databricks until both the producer and receiver work. This isolates Azure Event Hubs connectivity from later Spark/storage problems.

6. Databricks Serverless implementation and troubleshooting

6.1 Import stream_to_delta.py

36. Open Databricks Workspace.
37. Import databricks/stream_to_delta.py into your user/workspace folder.
38. Attach/select Serverless compute.
39. Replace placeholder namespace/path values before running.

6.2 Issue 2 - Databricks secret does not exist

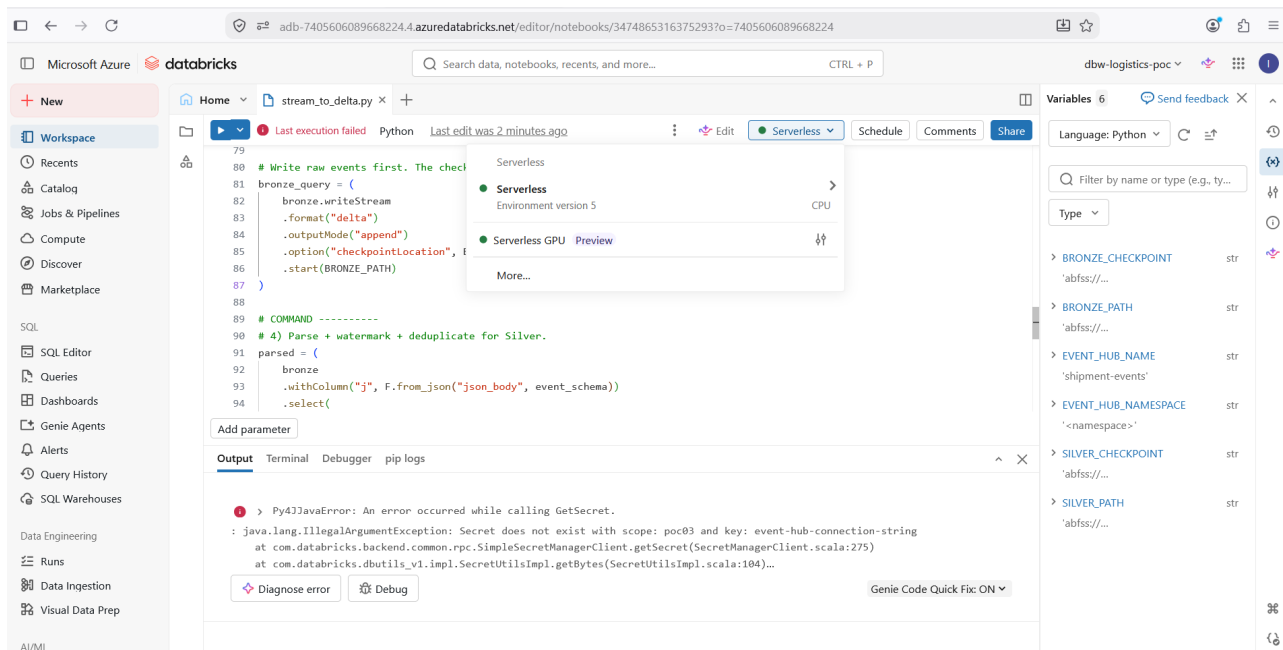


Figure 9 - stream_to_delta.py failed because secret scope poc03 / key event-hub-connection-string did not exist.

The notebook originally called:

```
EVENT_HUB_CONNECTION_STRING = dbutils.secrets.get(
    scope="poc03",
    key="event-hub-connection-string"
)
```

Two valid ways forward were identified in the run log:

- Preferred: create the Databricks secret scope/key and keep the connection string out of notebook source.
- Temporary POC fallback: paste the connection string into the notebook to continue troubleshooting, but never commit or share that notebook and remove/rotate the secret afterward.

Future-reference recommendation: Use the secret approach. The direct hard-coded credential path made later screenshots risky and should be treated only as a temporary debugging step.

6.3 Issue 3 - Serverless rejected fs.azure.account.key and showed JVM/config error

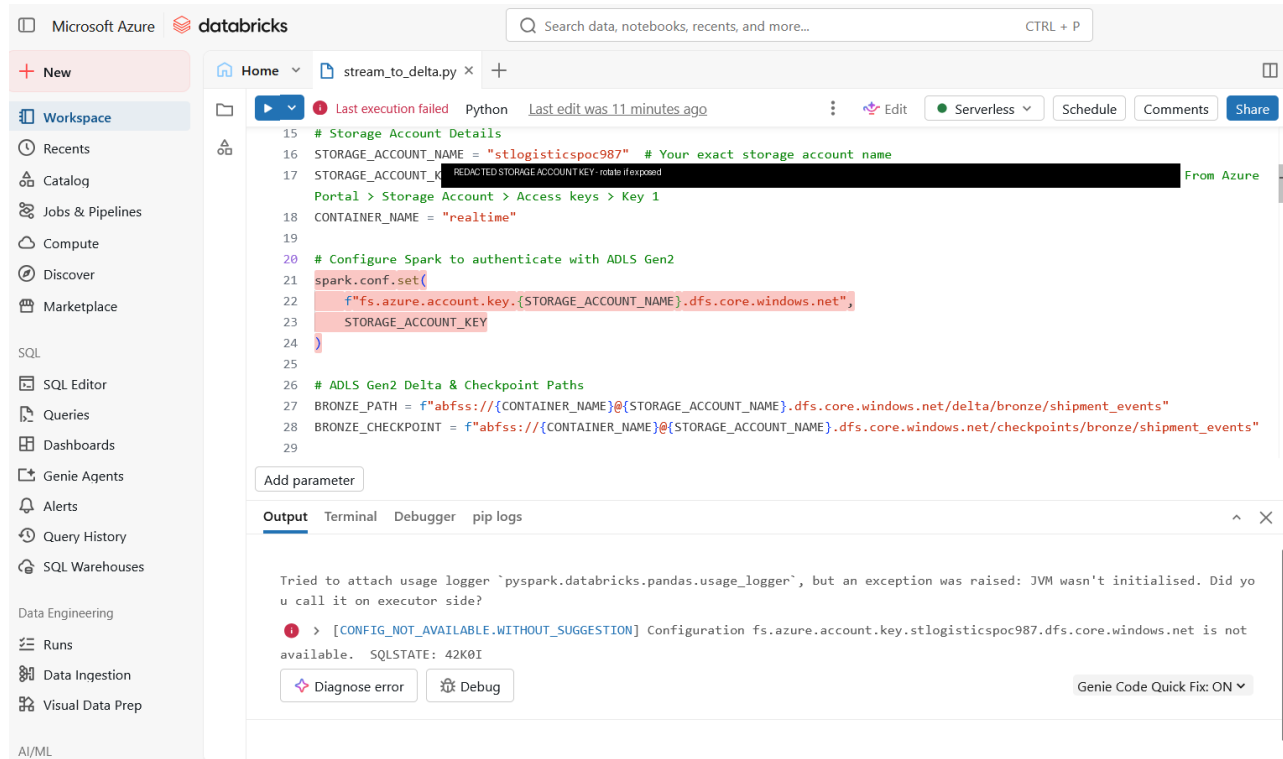


Figure 10 - Serverless failure: CONFIG_NOT_AVAILABLE for fs.azure.account.key. The exposed key line has been redacted.

The visible "JVM wasn't initialised" text was secondary. The primary error in the screenshot is that Serverless did not allow the storage account key Spark configuration.

The run log documented three approaches:

- Switch to an all-purpose/single-user cluster if you intentionally need spark.conf.set with a storage key.
- Use Unity Catalog storage credentials/external locations for external ADLS on Serverless.
- The successful POC path: use a Unity Catalog managed Volume and remove spark.conf.set entirely.

Final Serverless-compatible storage paths

```
# Successful Serverless storage base used in the run
spark.sql("CREATE VOLUME IF NOT EXISTS dbw_logistics_poc.default.realtime")
VOLUME_BASE = "/Volumes/dbw_logistics_poc/default/realtime"

BRONZE_PATH = f"{VOLUME_BASE}/delta/bronze/shipment_events"
SILVER_PATH = f"{VOLUME_BASE}/delta/silver/shipment_events"
BRONZE_CHECKPOINT = f"{VOLUME_BASE}/checkpoints/v3/bronze/shipment_events"
SILVER_CHECKPOINT = f"{VOLUME_BASE}/checkpoints/v3/silver/shipment_events"
```

Why the Azure storage container looked empty: Once the paths changed to /Volumes/..., the files were written to the Databricks managed Unity Catalog Volume. They were no longer being written to stlogisticspoc987/realtime in the standalone Azure storage account. This explains the empty external container observed during troubleshooting.

6.4 Issue 4 - infinite streaming trigger not supported on Serverless

The run log records [INFINITE_STREAMING_TRIGGER_NOT_SUPPORTED]. The fix was to use a finite available-now micro-batch.

Bronze available-now trigger

```
bronze_query = (
    bronze.writeStream
        .format("delta")
        .outputMode("append")
        .option("checkpointLocation", BRONZE_CHECKPOINT)
        .trigger(availableNow=True)
        .start(BRONZE_PATH)
)
bronze_query.awaitTermination()
```

Silver available-now trigger

```
silver_query = (
    silver.writeStream
        .format("delta")
        .outputMode("append")
        .option("checkpointLocation", SILVER_CHECKPOINT)
        .trigger(availableNow=True)
        .start(SILVER_PATH)
)
silver_query.awaitTermination()
```

Key behavior: availableNow=True is not a continuously running stream. It drains the data available for that run and terminates. Send a test batch first, then run the notebook, or rerun the notebook after new data arrives.

6.5 Issue 5 - 50 events were sent, but the pipeline initially showed 0 rows

The challenge log explains two contributing factors: the available-now query checked before new events were sent, and an existing checkpoint could make a later run skip offsets already considered. The recovery used a fresh checkpoint version and startingOffsets=earliest for the test.

```
# For the recovery/test run
BRONZE_CHECKPOINT = f'{VOLUME_BASE}/checkpoints/v3/bronze/shipment_events'
SILVER_CHECKPOINT = f'{VOLUME_BASE}/checkpoints/v3/silver/shipment_events'

kafka_options = {
    # ... other Kafka options ...
    "startingOffsets": "earliest",
    "failOnDataLoss": "false",
}
```

Run sequence used for the successful test:

40. Send 50 events locally: `python producer\send_events.py --count 50`
41. Clear notebook state/output if necessary.
42. Run `stream_to_delta.py` with the new v3 checkpoint path.
43. Inspect the query profile and Catalog Volume.

The screenshot displays the Databricks web interface. On the left, a sidebar contains navigation links: New, Workspace, Recents, Catalog (selected), Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. Below these are sections for Data Engineering (Runs, Data Ingestion, Visual Data Prep) and AI/ML.

The main panel is titled 'Catalog Explorer > dbw_logistics_poc > default > realtime'. It features a search bar and a 'Share' button. Under the 'Overview' tab, there's a section for 'Description' with an 'AI generate' button. Below this, a path is shown: `/Volumes/dbw_logistics_poc/default/realtime / delta / silver / shipment_events / _delta_log`, accompanied by 'Refresh' and 'Create directory' buttons.

A search filter 'Filter files and directories at this level' is present above a table of files:

Name	Size	Last modified
<code>00000000000000000000000000000000.crc</code>	2.76 KB	1 hour ago
<code>00000000000000000000000000000000.json</code>	1.88 KB	1 hour ago
<code>00000000000000000000000000000001.crc</code>	2.84 KB	1 hour ago
<code>00000000000000000000000000000001.json</code>	843.00 B	1 hour ago
<code>_tmp_path_dir</code>		
<code>_staged_commits</code>		

At the bottom, an 'About this volume' section indicates the owner is 'irshad alam'.

Page 14

7. Silver validation, Gold creation, and SQL serving

7.1 Issue 6 - first Silver validation returned 0 rows

The screenshot shows the Databricks interface with a notebook named 'inspect_and_validate.py' running. The code in the notebook is as follows:

```

1 # Databricks notebook source
2 from pyspark.sql import functions as F
3
4 SILVER_PATH = "/Volumes/dbw_logistics_poc/default/realtime/delta/silver/shipment_events"
5
6 df = spark.read.format("delta").load(SILVER_PATH)
7
8 print("Silver rows:", df.count())
9 print("Distinct event_id:", df.select("event_id").distinct().count())
10
11 display(
12     df.groupBy("event_type")
13         .count()
14         .orderBy(F.desc("count"))
15 )
16

```

The output of the notebook run is displayed in the 'Output' tab:

```

Silver rows: 0
Distinct event_id: 0

```

The 'Query wall-clock duration' is 200 ms. The 'Query Source' is 'inspect_and_validate.py'.

Figure 13 - Initial inspect_and_validate run showed Silver rows = 0 and distinct event_id = 0.

What this screenshot proves: Bronze ingestion had succeeded, but the Silver output was not yet populated in that validation attempt. This is why the later recovery notebook reparsed Bronze and wrote Silver before validating it.

7.2 Recovery - parse Bronze, deduplicate, write Silver, validate metrics

Recovery/validation pattern reconstructed from the successful run


```
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, StringType, LongType, DoubleType
```

```
VOLUME_BASE = "/Volumes/dbw_logistics_poc/default/realtime"
BRONZE_PATH = f'{VOLUME_BASE}/delta/bronze/shipment_events'
SILVER_PATH = f'{VOLUME_BASE}/delta/silver/shipment_events'
```

```
event_schema = StructType([
    StructField("event_id", StringType(), False),
    StructField("order_id", LongType(), False),
    StructField("event_type", StringType(), False),
    StructField("event_ts", StringType(), False),
    StructField("region", StringType(), False),
    StructField("revenue", DoubleType(), True),
    StructField("fulfillment_minutes", LongType(), True),
    StructField("producer_seq", LongType(), True),
])
```

```
df_bronze = spark.read.format("delta").load(BRONZE_PATH)
print("1. Total raw events in Bronze:", df_bronze.count())
```

```
df_parsed = (
    df_bronze
    .withColumn("j", F.from_json("json_body", event_schema))
    .select("j.*", "partition", "offset", "eventhub_enqueued_ts", "ingested_at")
    .withColumn("event_ts", F.to_timestamp("event_ts"))
    .filter(F.col("event_id").isNotNull())
    .dropDuplicates(["event_id"])
)
```

```
df_parsed.write.format("delta").mode("overwrite").save(SILVER_PATH)
print("2. Silver Delta table successfully populated!")
```

```
df_silver = spark.read.format("delta").load(SILVER_PATH)
print("3. Total clean events in Silver:", df_silver.count())
duplicates = df_silver.groupBy("event_id").count().filter("count > 1").count()
print("4. Duplicate count (must be 0):", duplicates)
```

The screenshot displays the Databricks workspace interface. On the left, the 'Workspace' sidebar shows a list of recent notebooks and queries. The main area shows a notebook titled 'inspect_and_validate.py' with the following code:

```
45 df_parsed.write.format("delta").mode("overwrite").save(SILVER_PATH)
46 print("2. Silver Delta table successfully populated!")
47
48 # 6. Validate Silver metrics
49 df_silver = spark.read.format("delta").load(SILVER_PATH)
50 print(f"3. Total clean events in Silver: {df_silver.count()}")
51
52 duplicates = df_silver.groupBy("event_id").count().filter("count > 1").count()
53 print(f"4. Duplicate count (must be 0): {duplicates}")
54
55 # 7. Display breakdown by event_type
56 display(
57     df_silver.groupBy("event_type")
58     .count()
59     .orderBy(F.desc("count"))
60 )
```

The 'Output' tab at the bottom shows the execution results:

```
> 3 dataframes: df_bronze, df_parsed, df_silver
1. Total raw events in Bronze: 50
2. Silver Delta table successfully populated!
3. Total clean events in Silver: 48
4. Duplicate count (must be 0): 0
```

Below the output, a 'Table' view is available, and a link to 'See performance (5)' is provided.

Figure 14 - Successful validation: Bronze = 50, Silver = 48, duplicate count = 0.

Why 50 became 48: The producer intentionally injects duplicate event IDs (for example every 25 events). The successful validation screenshot shows 50 raw Bronze events and 48 deduplicated Silver events, with duplicate count 0.

7.3 Build Gold Parquet KPI summary

export_gold_parquet.py - final volume-compatible logic

```
from pyspark.sql import functions as F

VOLUME_BASE = "/Volumes/dbw_logistics_poc/default/realtime"
SILVER_PATH = f"{VOLUME_BASE}/delta/silver/shipment_events"
GOLD_PATH = f"{VOLUME_BASE}/gold/shipment_summary"

df_silver = spark.read.format("delta").load(SILVER_PATH)
print(f"Loaded {df_silver.count()} clean events from Silver.")

gold_kpi_summary = (
    df_silver.groupBy("region")
    .agg(
        F.count("order_id").alias("total_events"),
        F.countDistinct("order_id").alias("unique_orders"),
        F.round(F.sum("revenue"), 2).alias("total_revenue"),
        F.round(F.avg("fulfillment_minutes"), 1).alias("avg_fulfillment_mins"),
        F.count(F.when(F.col("event_type") == "DELAYED", 1)).alias("delayed_shipments"),
        F.count(F.when(F.col("event_type") == "DELIVERED", 1)).alias("delivered_shipments"),
    )
    .withColumn(
        "on_time_delivery_pct",
        F.round(
            (F.col("delivered_shipments") /
             (F.col("delivered_shipments") + F.col("delayed_shipments"))) * 100,
            2,
        ),
    )
    .orderBy(F.desc("total_revenue"))
)

gold_kpi_summary.write.mode("overwrite").parquet(GOLD_PATH)
print("Gold KPI summary exported successfully!")
display(gold_kpi_summary)
```

The screenshot shows a Databricks workspace with a Python notebook named `export_gold_parquet.py`. The notebook code calculates various metrics from Silver events and exports a Gold KPI summary. The output shows 48 clean events loaded and the summary exported successfully.

```

19 F.count("order_id").alias("total_events"),
20 F.countDistinct("order_id").alias("unique_orders"),
21 F.round(F.sum("revenue"), 2).alias("total_revenue"),
22 F.round(F.avg("fulfillment_minutes"), 1).alias("avg_fulfillment_mins"),
23 F.count(F.when(F.col("event_type") == "DELAYED", 1)).alias("delayed_shipments"),
24 F.count(F.when(F.col("event_type") == "DELIVERED", 1)).alias("delivered_shipments"),
25 )
26 .withColumn(
27     "on_time_delivery_pct",
28     F.round(
29         (F.col("delivered_shipments") / (F.col("delivered_shipments") + F.col("delayed_shipments"))) * 100,
30         2,
31     ),
32 )
33 .orderBy(F.desc("total_revenue"))

```

The output shows 2 dataframes: `df_silver`, `gold_kpi_summary`. It indicates that 48 clean events were loaded from Silver and the Gold KPI summary was exported successfully.

region	total_events	unique_orders	total_revenue	avg_fulfillment_mins	delayed_shipments	delivered_shipments	on_time_delivery_pct
1.2	1.2	1.2	1.2	1.2	1.2	1.2	1.2

Figure 15 - Gold KPI summary exported successfully from 48 clean Silver events.

7.4 Create queryable Databricks SQL views

The screenshot evidence shows CREATE OR REPLACE VIEW statements using direct Delta/Parquet path syntax. Use these commands in Databricks SQL Editor:

Actual SQL pattern shown in the supplied SQL screenshot

```
CREATE OR REPLACE VIEW dbw_logistics_poc.default.silver_shipment_events AS
SELECT * FROM delta.`/Volumes/dbw_logistics_poc/default/realtime/delta/silver/shipment_events`;
```

```
CREATE OR REPLACE VIEW dbw_logistics_poc.default.gold_shipment_summary AS
SELECT * FROM parquet.`/Volumes/dbw_logistics_poc/default/realtime/gold/shipment_summary`;
```

The screenshot shows the Databricks SQL Editor with the following SQL queries:

```

1 -- 1. Create a View over the Silver Delta table
2 CREATE OR REPLACE VIEW dbw_logistics_poc.default.silver_shipment_events AS
3 SELECT * FROM delta.`/Volumes/dbw_logistics_poc/default/realtime/delta/silver/shipment_events`;
4
5 -- 2. Create a View over the Gold Parquet table
6 CREATE OR REPLACE VIEW dbw_logistics_poc.default.gold_shipment_summary AS
7 SELECT * FROM parquet.`/Volumes/dbw_logistics_poc/default/realtime/gold/shipment_summary`;

```

Figure 16 - Databricks SQL views created over Silver Delta and Gold Parquet.

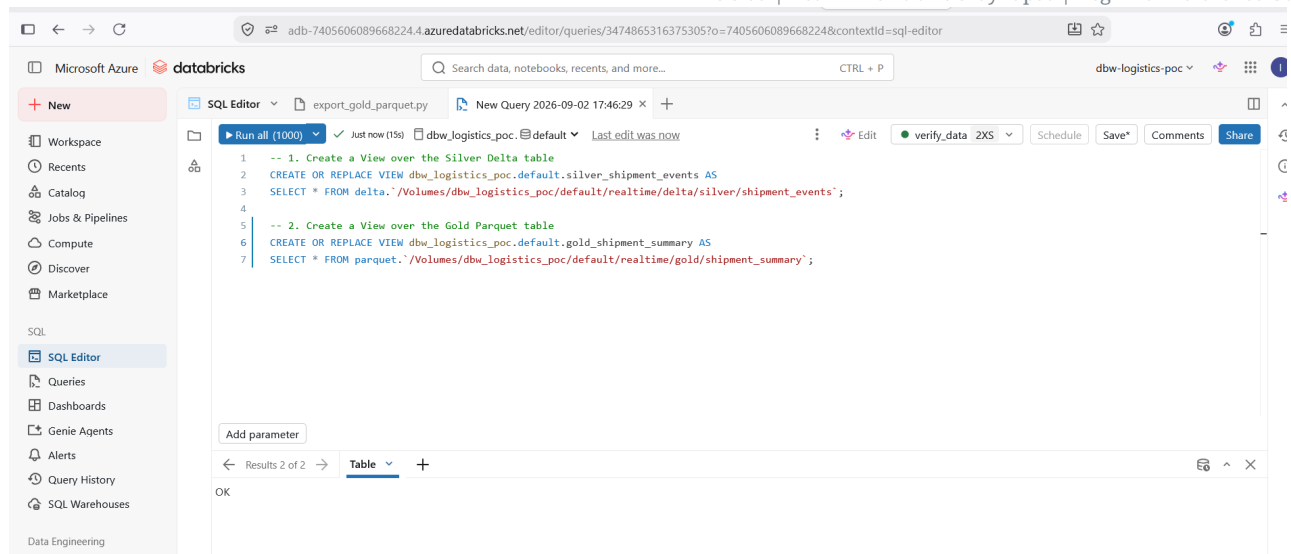


Figure 17 - Second supplied curated-data SQL capture. It shows the same view-creation screen and is retained because it was part of the evidence archive.

7.5 Verification queries

Gold KPI query

```
SELECT
  region,
  unique_orders,
  total_revenue,
  avg_fulfillment_mins,
  delayed_shipments,
  delivered_shipments,
  on_time_delivery_pct
FROM dbw_logistics_poc.default.gold_shipment_summary
ORDER BY total_revenue DESC;
```

Silver detailed verification query

```
SELECT
  event_type,
  region,
  count(*) AS event_count,
  round(avg(revenue), 2) AS avg_revenue,
  min(event_ts) AS earliest_event,
  max(event_ts) AS latest_event
FROM dbw_logistics_poc.default.silver_shipment_events
GROUP BY event_type, region
ORDER BY event_count DESC;
```

8. Synapse Serverless SQL reference and the ADLS requirement

Evidence boundary: The project contains a Synapse serverless SQL script, but the supplied screenshots do not show Synapse Studio execution. Also, the successful final Gold data was stored under a Databricks /Volumes path. Synapse cannot query that local Databricks Volume path directly; the Gold Parquet must be present in ADLS (or another storage endpoint Synapse can read).

8.1 If Gold Parquet is written to ADLS

Use the project path convention:

```
https://<storage-account>.dfs.core.windows.net/realtime/gold/shipment_summary/*.parquet
```

Then open Synapse Studio, connect to the built-in serverless SQL pool, and run:

`synapse/serverless_queries.sql` from the POC project

```
-- A. Smoke test
SELECT TOP 100 *
FROM OPENROWSET(
    BULK 'https://<storage-account>.dfs.core.windows.net/realtime/gold/shipment_summary/*.parquet',
    FORMAT = 'PARQUET'
) AS rows;

-- B. Read only needed columns
SELECT
    region,
    total_orders,
    delayed_shipments,
    revenue
FROM OPENROWSET(
    BULK 'https://<storage-account>.dfs.core.windows.net/realtime/gold/shipment_summary/*.parquet',
    FORMAT = 'PARQUET'
)
WITH (
    region VARCHAR(50),
    total_orders BIGINT,
    revenue FLOAT,
    delayed_shipments BIGINT
) AS gold
ORDER BY region;

-- C. Business totals
SELECT
    SUM(total_orders) AS total_orders,
    ROUND(SUM(revenue), 2) AS revenue,
    SUM(delayed_shipments) AS delayed_shipments
FROM OPENROWSET(
    BULK 'https://<storage-account>.dfs.core.windows.net/realtime/gold/shipment_summary/*.parquet',
    FORMAT = 'PARQUET'
)
WITH (
    total_orders BIGINT,
    revenue FLOAT,
    delayed_shipments BIGINT
) AS gold;
```

Schema note: The original Synapse script expects columns named `total_orders`, `revenue`, and `delayed_shipments`. The final Databricks Gold screenshot/code uses `unique_orders` and `total_revenue`. Before running Synapse, either align the Gold column names or update the SQL `WITH` clause to match the actual Parquet schema.

9. Fabric Eventstream / Eventhouse / KQL reference path

Reference-only section: These assets are present in the POC project and original plan, but no Fabric screenshot was supplied in screenshot(2).zip. Use this section for future completion/repractice; do not use the screenshots in this PDF as proof that Fabric was deployed.

9.1 Create KQL table

```
.create table ShipmentEvents (
  event_id:string,
  order_id:long,
  event_type:string,
  event_ts:datetime,
  region:string,
  revenue:real,
  fulfillment_minutes:long,
  producer_seq:long
)
```

9.2 KQL verification queries

```
ShipmentEvents
| take 20

ShipmentEvents
| summarize events=count() by bin(event_ts, 5m), event_type
| order by event_ts desc

ShipmentEvents
| where event_type == "DELAYED"
| summarize delayed=count() by region
| order by delayed desc

ShipmentEvents
| summarize latest_event=max(event_ts)
| extend freshness_seconds = datetime_diff("second", now(), latest_event)

ShipmentEvents
| summarize copies=count() by event_id
| where copies > 1
| order by copies desc
```

9.3 Fabric setup sequence

44. Create a Fabric workspace on an available Fabric capacity/trial.
45. Create an Eventhouse and KQL database.
46. Create ShipmentEvents using the KQL DDL above.
47. Create an Eventstream and add Azure Event Hubs as the source.
48. Select shipment-events and configure the connection/consumer settings.
49. Add Eventhouse/KQL as the destination and map JSON fields.
50. Publish the Eventstream.
51. Send a new local batch with send_events.py.

52. Run KQL arrival, grouping, delay, duplicate, and freshness queries.
53. Create a small Lakehouse/shortcut and semantic model if available.

10. Monitoring, failure recovery, evidence, and cleanup

10.1 What to monitor

- Event Hubs: Incoming Messages, Incoming Bytes, throttling/errors.
- Databricks: rows written/read, query duration, checkpoint paths, failures.
- Data freshness: max(event_ts) vs current UTC time.
- Silver quality: row count, distinct event_id count, duplicate count.
- Gold quality: KPI totals by region and latest event timestamp.
- Fabric/Synapse, when implemented: ingestion/query status and data processed.

10.2 Failure and recovery evidence captured in this POC

Failure	Working recovery	Evidence
Connection string malformed	Correct .env / copy Connection string-primary key	Figures 6-7
Databricks secret missing	Create secret or temporarily supply credential	Figure 9
Serverless storage-key config blocked	Use Unity Catalog Volume / governed storage path	Figure 10
Infinite stream unsupported	Use trigger(availableNow=True)	Run log
0 rows due timing/checkpoint	Send batch before run; new v3 checkpoint; earliest for recovery	Figures 11-13
Silver 0 rows	Reparse Bronze, dedup, overwrite Silver, validate	Figures 13-14

10.3 Cleanup

Azure CLI cleanup command from the run log

```
az group delete --name rg-logistics-poc --yes --no-wait
```

- Stop/suspend any Databricks SQL warehouse when you finish testing.
- Stop/terminate paid Databricks compute if applicable.
- Delete the POC resource group only when you have saved screenshots and outputs.
- Fabric resources may need separate cleanup because they are not necessarily deleted with the Azure resource group.
- Rotate any credential/key that was exposed in a screenshot, notebook, terminal history, or public repository.

11. Fast rerun checklist for future practice

- ☐ Activate Python environment and install requirements.
- ☐ Verify .env has a valid Event Hubs connection string and no placeholders.
- ☐ Run `send_events.py --count 20`.
- ☐ Run `receive_events.py` and confirm events arrive.
- ☐ Open Databricks; verify the realtime Volume exists.
- ☐ For availableNow: send the main batch before running the Databricks stream.

- ☐ Run `stream_to_delta.py` and confirm Bronze rows written.
- ☐ Run `inspect_and_validate.py`; expect Silver > 0 and duplicate count = 0.
- ☐ Run `export_gold_parquet.py`; confirm Gold KPI output.
- ☐ Create/refresh Databricks SQL views and run Gold/Silver queries.
- ☐ If testing Synapse, first ensure Gold Parquet is in ADLS, not only /Volumes.
- ☐ If testing Fabric, publish Eventstream and run KQL checks.
- ☐ Capture evidence and stop compute.

12. Issue-to-fix quick reference

Symptom	Root cause	Fix
[Errno 11001] getaddrinfo failed	.env still contains placeholder namespace/host	Replace placeholders with the actual Event Hubs namespace and connection string.
Connection string is either blank or malformed	Wrong field copied, .env not loaded, or bad formatting	Copy Connection string-primary key; verify .env filename and Python-read length.
Secret does not exist with scope poc03...	Databricks secret scope/key was never created	Create the secret or use a temporary secure fallback; never commit secrets.
CONFIG_NOT_AVAILABLE fs.azure.account.key...	Serverless blocks that Spark credential setting	Use Unity Catalog governed access or the managed Volume path; alternatively switch compute.
JVM wasn't initialised	Secondary message after unsupported Spark config	Fix the primary CONFIG_NOT_AVAILABLE error.
INFINITE_STREAMING_TRIGGER_NOT_SUPPORTED	Serverless interactive run does not allow the indefinite trigger used	Use <code>.trigger(availableNow=True)</code> .
No rows after <code>send_events.py</code>	<code>availableNow</code> started before data arrived and/or checkpoint state	Send events first, rerun with intended checkpoint; use a fresh checkpoint only for deliberate recovery testing.
Silver rows = 0	Silver output not populated in that run	Parse Bronze batch, deduplicate by <code>event_id</code> , write Silver, then validate.
Synapse cannot use /Volumes path	Databricks Volume is not an ADLS URL	Write/copy Gold to ADLS and run OPENROWSET against <code>dfs.core.windows.net</code> .

Appendix A. Full screenshot evidence index

Every supplied screenshot is included in the main body. This index maps the archive filename to its location/purpose. Sensitive credential content was redacted where necessary.

Archive filename	Figure	Meaning
1_resource_group_created.png	Figure 1	Azure resource group created
2_Event_Hubs_namespace.png	Figure 2	Event Hubs namespace deployed
3_storage_account.png	Figure 3	Storage account deployed
9_adls_gen2_storage_access_key.png	Figure 4	Storage access-key page; sensitive identity/key display protected
7_databricks_created.png	Figure 5	Databricks workspace deployed
4_error_send_events.png	Figure 6	Producer malformed connection-string error
5_success_send_events.png	Figure 7	Producer success
6_receive_events.png	Figure 8	Receiver success
8_databricks_stream_to_delta_error.png	Figure 9	Missing Databricks secret
10_databricks_jvm_error.png	Figure 10	Serverless Spark config/JVM error; secret redacted
11_databricks_stream_running_success.png	Figure 11	Successful Bronze available-now write
11_databricks_verify_data_catalog.png	Figure 12	Unity Catalog Volume files
12_inspect_and_validate_data.png	Figure 13	Initial Silver zero-row validation
13_validate_silver_metrics.png	Figure 14	Recovered Silver: 48 rows, 0 duplicates
14_export_gold_parquet_success.png	Figure 15	Gold KPI export success
15_verify_sql_data.png	Figure 16	Databricks SQL views
16_Curated_Data.png	Figure 17	Second curated SQL capture supplied in archive

Appendix B. Security and GitHub safety checklist

- Never commit .env.
- Never commit Event Hubs connection strings, storage keys, SAS tokens, client secrets, or Databricks tokens.
- Do not publish screenshots that show credentials.
- Use .env.example with placeholders only.
- Prefer managed identity / Unity Catalog credentials / Databricks secret scopes over inline keys.
- Rotate any credential that appeared in a screenshot or pasted notebook cell.
- Before pushing to GitHub, search the repository for Endpoint=sb://, SharedAccessKey=, AccountKey, SAS signatures, and long token strings.

Final verified milestone from supplied evidence: Event Hubs producer/receiver worked; Databricks Serverless ingested 50 raw events into the Unity Catalog Volume; the repaired Silver layer contained 48 clean unique events with 0 duplicates; Gold KPIs were exported; Databricks SQL views were created. Fabric/Synapse/Purview remain reference paths unless you have additional evidence not included in the supplied screenshot archive.